الأساليب الرشيقة (Agile Methods)

لماذا ظهرت الـ Agile Methods؟

- الأعمال الحديثة تعمل في بيئة **سريعة التغيير** ← لا يمكن استخلاص متطلبات مستقرة وكاملة مسبقاً.
- **المشكلة مع Plan-Driven (Waterfall):**
  - يحدد المتطلبات كاملة أولاً ← ثم يصمم ← ثم يبني ← ثم يختبر.
  - عند تغيّر المتطلبات ← **إعادة عمل كاملة (Rework)** ← تأخير وتكلفة عالية.
  - قد يُسلَّم البرنامج **بعد فوات الأوان** — السبب الأصلي لبنائه قد تغيّر كلياً!
- **السبب الجذري لظهور Agile:** عدم الرضا عن الأساليب الثقيلة (**Heavyweight approaches**) في أواخر التسعينيات.

---

1 الخصائص المشتركة لجميع Agile Methods (3 خصائص)

الخاصية	الشرح
Interleaved Processes	Specification + Design + Implementation تسير **معاً** وليس بالتسلسل — لا يوجد مواصفة تفصيلية مسبقة.
Incremental Development	النظام يُبنى على **دفعات صغيرة** — المستخدمون يُقيّمون كل دفعة ويقترحون تغييرات.
Extensive Tool Support	اعتماد كبير على الأدوات: Automated testing, Configuration management, UI automation.

---

2 الـ Agile Manifesto — الوثيقة الأساسية

صادرها كبار مطوري الـ Agile — تقول:
"نحن نكتشف طرقاً أفضل لتطوير البرمجيات بالممارسة والمساعدة... وتوصلنا إلى تقدير:"

القيم الأربع (الأهم للـ MCQ):

(الأيسر دائماً أهم من الأيمن)

- **الأفراد والتفاعلات** (Interactions & Individuals) > العمليات والأدوات (Tools & Processes).
- **برمجيات تعمل** (Working Software) > التوثيق الشامل (Comprehensive Documentation).
- **التعاون مع العميل** (Customer Collaboration) > التفاوض على العقود (Contract Negotiation).
- **الاستجابة للتغيير** (Responding to Change) > اتباع خطة (Following a Plan).

⚠ **نقطة MCQ خطيرة:** الـ Manifesto لا يقول تجاهل الجانب الأيمن — بل يقول **الأيسر أهم**.

---

3 المبادئ الخمسة للـ Agile

1. **Customer Involvement:** العميل مشارك طوال العملية — يحدد الأولويات ويُقيّم كل دفعة.
2. **Embrace Change:** توقَّع تغيّر المتطلبات وصمِّم النظام ليستوعب التغيير.
3. **Incremental Delivery:** تسليم النظام على دفعات — العميل يحدد متطلبات كل دفعة.
4. **Maintain Simplicity:** البساطة في الكود وفي العملية — اعمل على إزالة التعقيد بنشاط.
5. **People, not Process:** مهارات الفريق مهمة — اتركهم يطوّرون أسلوبهم الخاص دون فرض عمليات صارمة.

4 Agile vs Plan-Driven — الفرق الجوهري

المعيار	Plan-Driven	Agile
التكرار (Iteration)	داخل كل نشاط بشكل مستقل.	عبر الأنشطة معاً.
التواصل	وثائق رسمية بين المراحل.	تواصل غير رسمي ومباشر.
المتطلبات والتصميم	تتطور بشكل **منفصل**.	تتطور **معاً** في نفس الوقت.
الوثائق	كثيرة ورسمية.	أقل ما يمكن.
التخطيط	مسبق وكامل.	تدريجي ومستمر.

في الواقع العملي: كثير من المشاريع تخلط بين الأسلوبين — ليس بالضرورة Agile خالص أو Plan-Driven خالص.

5 متى تنجح Agile Methods؟ (نوعان من المشاريع)

- **Product Development:** شركة تطور منتجاً صغيراً أو متوسطاً للبيع - **تقريباً كل التطبيقات والمنتجات الحالية** تُطوَّر بـ Agile.
- **Custom System Development:** تطوير داخل منظمة، العميل **ملتزم بالمشاركة**، قليل من المتطلبات الخارجية والقواعد التنظيمية.

لماذا تنجح هنا؟

- يمكن التواصل المستمر بين العميل والفريق.
- النظام مستقل وليس متشابكاً مع أنظمة أخرى تُطوَّر بالتوازي.
- الفريق **صغير في نفس الموقع** ← التواصل الغير رسمي يعمل بكفاءة.

نقاط MCQ احفظها جيداً

- ◈ **Agile** ظهر بسبب عدم الرضا عن **Heavyweight Plan-Driven approaches** في أواخر التسعينيات.
- ◈ أمثلة Agile methods: **XP, Scrum, DSDM**.
- ◈ **3 خصائص مشتركة** لكل Agile: Interleaved + Incremental + Tool Support.
- ◈ Agile يُصدر نسخاً كل **أسبوعين أو ثلاثة**.
- ◈ **القيم الأربع** لـ Agile Manifesto — الأيسر دائماً أهم من الأيمن.
- ◈ أهم قيمة: **Working Software** أهم من Comprehensive Documentation.
- ◈ **5 مبادئ:** Customer Involvement, Embrace Change, Incremental Delivery, Simplicity, People not Process.
- ◈ في Agile: التكرار **عبر الأنشطة** | في Plan-Driven: التكرار **داخل كل نشاط**.
- ◈ Agile مناسب لـ: **Custom development** + **Product development** بمشاركة العميل.
- ◈ Agile غير مناسب لـ: **Safety-critical systems** التي تحتاج تحليل كامل مسبقاً.

تقنيات التطوير الرشيق (Agile Development Techs)

المقدمة: البرمجة القصوى (Extreme Programming - XP)

القسم مبني على ممارسات XP، وهي من أكثر الأساليب الرشيقة تأثيراً.

- **المؤسس:** Kent Beck عام 1998.
- **لماذا سميت "Extreme"؟** لأنه أخذ الممارسات الجيدة إلى أقصى درجاتها (مثل دمج واختبار عدة نسخ في يوم واحد).
- **في الواقع:** الشركات نادراً ما تطبقه كاملاً، بل تختار ما يناسبها وتجمعه عادةً مع Scrum.

ممارسات XP الـ 10 (للحفظ كاملاً)

الممارسة	الشرح المختصر
Collective Ownership	أي مطور يعمل في كل أجزاء النظام؛ أي شخص يمكنه تغيير أي كود.
Continuous Integration	دمج المهمة فور اكتمالها في النظام، ويجب نجاح جميع الـ Unit Tests.
Incremental Planning	المتطلبات على "Story Cards"؛ يقسمها المطور لـ Tasks حسب الوقت والأولوية.
On-site Customer	ممثل العميل متواجد بدوام كامل مع الفريق لتوفير المتطلبات.
Pair Programming	المطورون يعملون في أزواج على نفس الجهاز؛ يراجع كل منهم عمل الآخر.
Refactoring	تحسين الكود باستمرار فور رؤية أي فرصة للتحسين.
Simple Design	تصميم ما يلبي المتطلبات الحالية فقط ولا أكثر.
Small Releases	تطوير أصغر مجموعة وظائف مفيدة أولاً، ثم إضافات متكررة.
Sustainable Pace	العمل الإضافي (Overtime) مرفوض؛ لأنه يقلل الجودة والإنتاجية.
Test-First Development	كتابة الاختبارات قبل كتابة الكود باستخدام أُطر اختبار آلية.

---

1 — قصص المستخدم (User Stories)

التعريف: سيناريو استخدام يصف تجربة يعيشها مستخدم للنظام، يُكتب على (Story Card).

- **الفلسفة:** لا يوجد نشاط منفصل للمتطلبات؛ بل تدمج مع التطوير مباشرة.

مراحل العمل:

1. يكتب العميل والفريق **Story Card**.
2. يقسمها الفريق إلى **Task Cards**.
3. تقدير الجهد والموارد لكل مهمة.
4. يحدد العميل الأولوية.
5. التنفيذ في نسخة مدتها أسبوعان تقريباً.

- ✓ **المزايا:** تفاعل الناس معها أسهل من الوثائق التقليدية، وتشجع المستخدمين على المشاركة.
- ✗ **المشكلات:** صعوبة التحقق من الاكتمال (Completeness)، وتجاهل الخبراء للتفاصيل البديهية.

---

2 — إعادة الهيكلة (Refactoring)

التعريف: البحث المستمر عن تحسينات في الكود وتطبيقها فوراً.

- **السبب:** التطوير التزايدي "يُدهور" هيكل الكود ويجعله معقداً ومكرراً.
- **أمثلة:** إزالة الكود المكرر، إعادة تسمية الـ Methods، تنظيف الـ Attributes.

⚠ **الفرق مع التقليدي:** في التقليدي "تصمم للتغيير مسبقاً"، أما في XP فالقاعدة هي "Refactor باستمرار" لأن التوقعات نادراً ما تتحقق.

---

3 — الاختبار أولاً (Test-First Development)

التعريف: كتابة الاختبار ثم كتابة الكود ليجتازه. تطور لاحقاً إلى **TDD** (Test-Driven Development).

الخصائص الأربعة:

1. **Test-First:** الاختبار يسبق الكود.
2. **Incremental:** الاختبارات تُبنى تدريجياً من القصص.
3. **User involvement:** مشاركة العميل في Acceptance Tests.
4. **Automated frameworks:** استخدام أدوات مثل **JUnit**.

- ✓ **المزايا:** تعريف السلوك المطلوب مسبقاً، تقليل سوء الفهم، وتجنب تأخر الاختبار عن التطوير (**Test-Lag**).
- ✗ **المشكلات:** تفضيل المبرمجين للكود على الاختبار، وصعوبة اختبار واجهات المستخدم (UI).

---

4 — البرمجة الثنائية (Pair Programming)

التعريف: مطوران على نفس الجهاز، وتتغير الأزواج ديناميكياً.

✓ المزايا:

1. **Collective Ownership:** يدعم "البرمجة بلا غرور" (Egoless Programming)؛ الكود ملك للجميع.
2. **Informal Review:** مراجعة فورية لكل سطر (أرخص من المراجعة الرسمية).
3. **Encourages Refactoring:** التحسين ينفع الجميع فوراً فيدعمونه.

- ✗ **الجدل:** شكوك حول الإنتاجية (دراسة Williams و Arisholm)، لكن فائدتها العظمى في **تقاسم المعرفة** لتقليل خطر مغادرة أعضاء الفريق.

مقارنة التقنيات الأربعة

التقنية	الهدف الأساسي	المشكلة الرئيسية
User Stories	التعبير عن المتطلبات بشكل بشري.	صعوبة التحقق من الاكتمال.
Refactoring	منع تدهور هيكل الكود.	ضغط الجداول الزمنية.
Test-First	ضمان الجودة وتجنب Test-Lag.	تفضيل البرمجة على الاختبار.
Pair Programming	مراجعة غير رسمية وتقاسم معرفة.	جدل حول الإنتاجية.

نقاط MCQ احفظها جيداً

- ◈ **XP** اخترعه Kent Beck عام 1998.
- ◈ **User Story** تُكتب على **Story Card** وتصف تجربة المستخدم.
- ◈ المشكلة الكبرى للقصص هي **Completeness** (الاكتمال).
- ◈ **Refactoring** يعني تحسين الكود فوراً؛ وXP يرفض مبدأ "Design for Change" التقليدي.
- ◈ **Test-First** يمنع الـ **Test-Lag** (تأخر الاختبار).
- ◈ **JUnit** هو إطار اختبار آلي شهير لـ Java.
- ◈ **Collective Ownership** تعني أن أي مطور يمكنه تعديل أي كود.
- ◈ **Pair Programming** فوائده: ملكية جماعية، مراجعة غير رسمية، وتشجيع الـ Refactoring.

إدارة المشاريع الرشيقة — SCRUM

1 مصطلحات Scrum السبعة (للحفظ)

المصطلح	التعريف المختصر
Development Team	مجموعة تنظم نفسها (7 أشخاص كحد أقصى) مسؤولة عن التنفيذ.
Potentially Shippable	منتج ناتج عن كل Sprint؛ جاهز للشحن نظرياً ولا يحتاج لعمل إضافي.
Product Backlog	قائمة شاملة لكل المهام (متطلبات، قصص، تحسينات، وثائق).
Product Owner	يحدد المتطلبات والأولويات ويراجع الـ Backlog (يمثل العميل).
Scrum (Meeting)	اجتماع يومي قصير لمراجعة التقدم وتحديد عمل اليوم.
ScrumMaster	**ميسّر** يضمن تطبيق العملية ويحمي الفريق؛ (ليس مدير مشروع).
Sprint	دورة تطوير واحدة مدتها **2 إلى 4 أسابيع**.
Velocity	مقياس لكمية العمل التي ينجزها الفريق في Sprint واحد (أساس التخطيط).

2 دورة الـ Sprint — خطوة بخطوة

1. **Product Backlog:** البداية من قائمة المهام الكلية.
2. **Prioritization:** الـ Product Owner يحدد الأولويات.
3. **Sprint Selection:** الفريق يختار مهام تناسب سرعته (**Velocity**).
4. **Sprint Backlog:** تحديد المهام الخاصة بالدورة الحالية.
5. **Execution (Sprint):** مدته 2-4 أسابيع، يتضمن:
   - **Daily Scrum:** اجتماع يومي للمراجعة.
   - **Scrum Board:** لوحة بيضاء مشتركة لمتابعة العمل.
6. **Sprint Review Meeting:** (نهاية الدورة) لتحسين العملية وتقديم تغذية راجعة للـ Backlog.
7. **Increment:** الخروج بمنتج جاهز للشحن (**Potentially Shippable**).

3 تفاصيل تقنية مهمة (نطاق الـ MCQ)

- **قاعدة الوقت:** الـ Sprint **لا يُمدد أبداً**؛ المهام غير المكتملة تعود للـ Backlog.
- **الشفافية:** الـ **Scrum Board** متاحة للجميع (أوراق ملاحظات على لوحة)، وأي شخص يمكنه تعديلها.
- **اجتماع المراجعة (Review Meeting):** له هدفان: (1) تحسين طريقة العمل، (2) تحديث الـ Backlog للدورة القادمة.

4 الاجتماع اليومي (Daily Scrum)

يجيب كل عضو على 3 أسئلة:

1. ماذا أنجزتُ منذ الاجتماع الأخير؟
2. ما المشاكل التي ظهرت؟
3. ماذا أخطط لفعله اليوم؟

⚠ **تنبيه:** الفريق يخطط لنفسه جماعياً، ولا يوجد توجيه من الأعلى (**No Top-Down direction**).

---

5 مزايا نظام Scrum

1. تقسيم المنتج لأجزاء مفهومة للمساهمين.
2. المتطلبات غير المستقرة لا توقف العمل.
3. رؤية كاملة للفريق تحسن التواصل والروح المعنوية.
4. العميل يرى تسليماً في الموعد ولا يُفاجأ في النهاية.
5. بناء ثقافة الثقة بين العميل والمطورين.

---

6 الـ Scrum الموزع (Distributed Scrum)

عندما يكون الفريق في مواقع مختلفة، نحتاج إلى:

- **Video Conferencing:** بين الـ Product Owner والفريق.
- **ScrumMaster محلي:** يجب أن يكون في نفس موقع الفريق.
- **تواصل فوري:** (Instant messaging / Video calls).
- **بيئة موحدة:** (Continuous Integration + Common Dev Environment).

مقارنة: ScrumMaster مقابل مدير المشروع (Project Manager)

المعيار	ScrumMaster	Project Manager
الطابع	ميسِّر (Facilitator)	مدير (Director)
التخطيط	قصير المدى (Sprint بـ Sprint)	طويل المدى ومسبق
السلطة	لا يأمر الفريق (خادم للقائد)	يوجه ويأمر الفريق
التقارير	للإدارة العليا (في الواقع)	للإدارة العليا

توسيع نطاق الـ Agile (Scaling Agile Methods)

المقدمة: ماذا يعني Scaling؟

الـ Agile صُمّم أصلاً لفرق صغيرة في نفس المكان — لكن الشركات الكبيرة تحتاجه أيضاً.

التوسيع له وجهان:

الوجه	المعنى
Scaling Up	توسيع الـ Agile ليخدم أنظمة كبيرة تحتاج أكثر من فريق صغير.
Scaling Out	نشر الـ Agile في فرق متعددة داخل شركة كبيرة ذات ممارسات راسخة.

---

1 المشاكل العملية مع Agile (القسم 3.4.1)

يواجه الـ Agile في الأنظمة الكبيرة طويلة الأمد 3 مشاكل رئيسية:

أ- المشكلة الأولى: العقود (Contractual Issues)

- لا يوجد مستند متطلبات نهائي لإدراجه في العقد.
- **الحل:** عقود تدفع بـ **الوقت المُستهلك (time-based)** وليس بالمتطلبات المُحددة.
- **الخطر:** عند نشوء مشاكل ← خلافات على من يتحمل التكلفة الإضافية.

ب- المشكلة الثانية: الصيانة (Maintenance Issues)

المشكلة	الشرح
غياب التوثيق	تقليل الوثائق يؤدي لصعوبة فهم النظام لاحقاً عند تغييره.
استمرارية مشاركة العميل	العميل قد لا يتوفر بشكل مستمر أثناء الصيانة عكس مرحلة التطوير.
استمرارية الفريق	تفرّق الفريق يعني فقدان المعرفة الضمنية وصعوبة الفهم للأعضاء الجدد.

ج- المشكلة الثالثة: الفرق الموزعة (Distributed Teams)

الـ Agile يعتمد على فرق صغيرة في نفس الموقع، بينما الواقع الحالي يفرض فرقاً موزعة حول العالم.

---

2 Agile مع Plan-Driven معاً (القسم 3.4.2)

في الواقع العملي: معظم المشاريع الكبيرة تجمع بين الاثنين.

المعايير التي تحدد الاختيار (متى نختار Plan-Driven؟)

المحور	السؤال	الجواب الذي يميل نحو Plan-Driven
النظام	كم حجم النظام؟	كبير
النظام	ما نوع النظام؟	Real-time / Safety-Critical
النظام	ما عمر النظام المتوقع؟	طويل (يحتاج وثائق أكثر)
النظام	هل هو خاضع لرقابة خارجية؟	نعم (مثل FAA)
الفريق	مستوى مهارة الفريق؟	منخفض
الفريق	كيف الفريق منظم؟	موزع / Outsourced
الفريق	ما الأدوات المتاحة؟	IDE ضعيفة (تحتاج وثائق أكثر)
المنظمة	هل هناك عقد بمواصفة مفصلة؟	نعم
المنظمة	هل العميل متاح للمشاركة؟	لا

---

3 Agile للأنظمة الكبيرة (القسم 3.4.3)

6 عوامل تجعل الأنظمة الكبيرة أصعب:

1) **System of Systems:** نظام مكون من أنظمة منفصلة تواصلية (فرق مختلفة لكل جزء).
2) **Brownfield Development:** التفاعل مع أنظمة قديمة موجودة (متطلبات غير مرنة).
3) **System Configuration:** كثير من العمل هو "تكوين" وليس كتابة كود جديد.
4) **Regulatory Constraints:** قواعد خارجية تفرض أنواعاً معينة من الوثائق.
5) **Prolonged Procurement:** مدة التطوير الطويلة تؤدي لتغيير وتفرق الفرق.
6) **Diverse Stakeholders:** مستخدمون متنوعون بأهداف مختلفة يصعب إشراكهم جميعاً.

إطارا العمل للأنظمة الكبيرة:

- **SAFe (Scaled Agile Framework):** طوّره **Dean Leffingwell**؛ إطار لتطوير البرمجيات بفرق متعددة.
- **ASM (Agile Scaling Model):** طوّرته **IBM**؛ نموذج تدريجي يمر بـ 3 مراحل.

مراحل IBM ASM الثلاث:

1. **Core Agile Development:** فرق تنظم نفسها، تركيز على البناء.
2. **Disciplined Agile Delivery:** تكيّف الممارسات، دمج المتطلبات والتصميم المعماري.
3. **Agility at Scale:** التعامل مع الفرق الموزعة، البيئات القديمة، والامتثال التنظيمي.

قواعد Agile للأنظمة الكبيرة (5 قواعد أساسية):

- لا يمكن الاعتماد الكامل على متطلبات تزايدية (بعض العمل المسبق ضروري).
- لا يمكن وجود Product Owner واحد فقط (ممثلون لكل جزء).
- لا يمكن التركيز على الكود فقط (التصميم المعماري وتوثيق قواعد البيانات ضروري).
- تصميم آليات تواصل بين الفرق (Video calls, Wiki, Instant messaging).
- لا يمكن عمل Continuous Integration الكامل (لكن يجب البناء بشكل متكرر).

الـ Scrum للفرق المتعددة (Multi-Team Scrum):

- **Role Replication:** كل فريق له Product Owner و ScrumMaster خاص به.
- **Product Architects:** تعاون المهندسين المعماريين لتصميم الـ Architecture الكلي.
- **Release Alignment:** تناسق مواعيد تسليم جميع الفرق لإنتاج نظام كامل.
- **Scrum of Scrums:** اجتماع يومي يضم ممثلاً عن كل فريق لمناقشة التقدم والمشاكل.

---

4 Agile على مستوى المنظمة (القسم 3.4.4)

4 تحديات تواجه الشركات الكبيرة:

1. **مقاومة المديرين:** عدم الرغبة في المخاطرة بأسلوب مجهول النتائج.
2. **معايير الجودة والأدوات:** إجراءات الشركة غير متوافقة مع Agile.
3. **تفاوت مستويات المهارة:** Agile ينجح أكثر مع الفرق عالية المهارة.
4. **المقاومة الثقافية:** التعارض بين العمليات التقليدية والرشاقة.

⚠ **مثال للتعارض:** في إدارة التغيير، تطلب الشركة موافقة مسبقة، بينما في الـ Agile يحسن المطور الكود (**Refactoring**) دون موافقة خارجية.

نقاط MCQ احفظها جيداً

- ◈ **Scaling Up** = أنظمة كبيرة | **Scaling Out** = منظمات كبيرة.
- ◈ **عقود Agile:** تعتمد على الدفع مقابل **الوقت** وليس المواصفات المحددة.
- ◈ **3 مشاكل صيانة:** غياب التوثيق، استمرارية مشاركة العميل، واستمرارية الفريق.
- ◈ **SAFe** طوّره Dean Leffingwell | **ASM** طوّره IBM.
- ◈ **Scrum of Scrums:** اجتماع يومي يضم ممثلاً عن كل فريق.
- ◈ **أفضل طريقة للتبني:** البدء بمجموعة متحمسة في مشروع ناجح واحد ثم النشر التدريجي.
- ◈ **التسمية (Agile/Plan-Driven):** ليست مهمة؛ المهم اختيار الأنسب لنوع النظام.